

Sjednocení dvou posloupností čísel

Dokumentace k programu

Radek Modrůczki

12. února 2026

Abstrakt

Program řeší úlohu nalezení sjednocení dvou celočíselných posloupností. Namísto využití vestavěných množinových operací a kontejnerů typu `set` je algoritmus postaven na setřídění spojených dat a následné eliminaci duplicit v jednom průchodu. Výsledná aplikace nabízí matematický výsledek a vhled do třídícího a vyhledávacího mechanismu.

Obsah

1	Přesné zadání	2
2	Zvolený algoritmus	2
3	Diskuse výběru algoritmu	2
4	Program	2
4.1	Použité datové struktury	2
4.2	Hlavní třídy a podprogramy	3
4.3	Tok řízení	3
5	Alternativní programová řešení	3
5.1	Využití standardních množinových typů	3
5.2	„Merge“ strategie	3
6	Reprezentace vstupních dat a jejich příprava	3
6.1	Reprezentace vstupních dat	3
6.2	Příprava vstupních dat	4
7	Reprezentace výstupních dat a jejich interpretace	4
7.1	Reprezentace výstupních dat	4
7.2	Interpretace výstupních dat	4
8	Průběh práce	4
9	Možná vylepšení	4

1 Přesné zadání

Na vstupu jsou dvě neuspořádané posloupnosti s různým počtem celočíselných prvků. Napište funkci, která nalezne jejich sjednocení, tj. unikátní prvky v obou posloupnostech, vytvoří z nich novou posloupnost a vytiskne ji. Obě posloupnosti bude nutné setřídít. Pro nalezení duplicit nepoužívejte kontejnery typu `set` ani další knihovny podporující vestavěné množinové operace.

2 Zvolený algoritmus

Pro nalezení sjednocení dvou neuspořádaných posloupností s různým počtem celočíselných prvků byl zvolen algoritmus, který seřadí a následně projde prvky ve dvou řádcích textového souboru.

Algoritmus nepoužívá žádné množinové operace. Proces začíná sloučením obou vstupních řádků do jednoho lineárního seznamu, který je následně vzestupně seřazen. Následně se sekvenčně filtrují duplicity, tedy algoritmus projde seřazené pole v jednom cyklu a do výsledné posloupnosti vloží prvek pouze tehdy, pokud se hodnotou liší od prvku bezprostředně předcházejícího. Tříděním byly duplicity seřazené do souvislých bloků vedle sebe, tímto je umožněno jednoduché porovnání sousedů, čímž jsou eliminovány opakující se hodnoty, a je vytvořena nová, vzestupně seřazená množina.

3 Diskuse výběru algoritmu

Při návrhu programu byl zvažován naivní přístup (Brute Force), který by pro každý prvek ověřoval jeho výskyt ve výsledném poli prostým lineárním vyhledáváním. Tato varianta byla zavrhnuta z důvodu neefektivní časové složitosti, která by při větším objemu dat způsobila zásadní zpomalení aplikace.

Řešení pomocí hashovacích tabulek nebylo možné implementovat vzhledem k zadání aplikace. Zvolená metoda využívající předběžné setřídění se tedy jeví jako optimální řešení. Snižuje výpočetní náročnost a zároveň plní požadavek na uspořádaný výstup, čímž eliminuje nutnost volat třídící funkci v separátním kroku.

4 Program

Aplikace je implementována v jazyce Python s důrazem na objektově orientovaný přístup. Je striktně odlišena správa dat od samotné algoritmicke logiky sjednocení.

4.1 Použité datové struktury

Pro uložení vstupních i výstupních dat jsou využity dynamické seznamy (typ `list`).

- **Vstupní buffery:** Dvě nezávislá pole uchovávají celá načtená čísla. Byla zvolena pro svou schopnost měnit velikost podle délky vstupu a pro efektivní podporu indexace.
- **Výstupní pole:** Slouží k akumulaci výsledných unikátních hodnot.
- **Reprezentace dat:** Všechna data jsou uložena jako celá čísla, přičemž převod textového formátu probíhá okamžitě při načítání, což zjednodušuje matematické porovnávání.

4.2 Hlavní třídy a podprogramy

Veškerá funkcionálnita je zapouzdřena ve třídě `UnionFinder`. Toto řešení umožňuje sdílení dat prostřednictvím vnitřního stavu objektu.

- `load_from_file(filename)` – zajišťuje vstupní operace, otevírá textový soubor a provádí parsování řádků na čísla. Plní interní seznamy `sequence_a` a `sequence_b`. Pokud soubor neexistuje nebo má špatný formát, je vyvolána výjimka, kterou zachytí hlavní program.
- `compute_union()` – provádí sloučení obou vstupních polí do jednoho dočasného seznamu. Následně tento seznam setřídí (metoda `sort()`) a v lineárním průchodu vyfiltruje duplicity porovnáváním sousedních prvků. Výsledek uloží do `result_union`.
- `print_result()` – slouží k prezentaci dat. Formátuje výsledný seznam do čitelné podoby, oddělí čísla čárkou a vypíše do konzole.

4.3 Tok řízení

Program je řízen blokem `main`, který celý proces provádí v lineárním sledu. Nejprve probíhá inicializace vytvořením instance třídy `UnionFinder`, poté načtení, kdy je voláno `load_from_file` a dochází k validaci existence souboru. Zpracování probíhá pomocí `compute_union`, kde je soubor tříděn. Výstup zajišťuje `print_result` pro zobrazení hotového sjednocení uživateli.

5 Alternativní programová řešení

Při vytváření aplikace bylo zvažováno několik odlišných přístupů k implementaci. Níže jsou uvedeny varianty, které byly v procesu zamítnuty, a důvody, které k zamítnutí vedly.

5.1 Využití standardních množinových typů

Toto řešení se jevilo jako nejpřirozenější způsob – konverze obou seznamů na datový typ `set` a následné použití operátoru sjednocení. Toto by řešení extrémně zjednodušilo, kód by mohl být i v rozsahu několika řádků. Bylo však explicitně zakázáno v zadání.

5.2 „Merge“ strategie

Bylo také zvažováno nejdříve setřídít posloupnost A, poté setřídít posloupnost B a následně provést jejich sloučení podobným způsobem, jako funguje řazení Merge Sort. Tato metoda by však zbytečně zvýšila složitost kódu a zvýšila pravděpodobnost zanesení chyb.

6 Reprezentace vstupních dat a jejich příprava

6.1 Reprezentace vstupních dat

Program pracuje výhradně s obyčejným textem. Aby aplikace správně fungovala a nebyla ukončena některou z možných chyb, je nutné dodržet určitá pravidla pro formátování.

Program očekává jako vstup obyčejný textový soubor, který musí obsahovat přesně dva řádky, které určují dvě vstupní množiny. Čísla ve vstupním souboru musí být oddělena mezerou a musí se jednat pouze o celá čísla. Není možné používat čárky, středníky ani tabulátory pro oddělení. Text také nesmí obsahovat žádná písmena ani speciální znaky.

6.2 Příprava vstupních dat

Pro vytvoření vstupních dat je ideální obyčejný textový editor, jako např. Poznámkový blok ve Windows. Je nutné, aby po vytvoření textového souboru byl uložen do stejné složky jako samotný program, jinak jej aplikace nenajde a dojde k chybě.

Obsah souboru musí tvořit přesně dva řádky: na prvním řádku první posloupnost a na druhém řádku druhá posloupnost. Jednotlivá čísla od sebe musí být oddělena mezerou. V souboru se nesmí vyskytovat žádné jiné oddělovače, písmena ani prázdné řádky navíc. V případě porušení těchto pravidel program nebude fungovat.

7 Reprezentace výstupních dat a jejich interpretace

7.1 Reprezentace výstupních dat

Výsledky programu se vypisují formou textových zpráv přímo na obrazovku v příkazovém řádku, hned pod příkaz, kterým uživatel program spustil. Výstup je strukturován do tří oddělených bloků pro lepší čitelnost.

První řádek informuje o počtu čísel, která byla načtena ze vstupního souboru. Následuje výpis výsledné seřazené posloupnosti, kde jsou čísla oddělena čárkou. Výpis je ukončen informací na posledním řádku, která uvádí počet nalezených unikátních prvků.

7.2 Interpretace výstupních dat

První řádek ve výstupu slouží jako kontrola správného čtení vstupního souboru. V případě nulové hodnoty je vstupní soubor prázdný nebo chybně formátovaný a program jej nemohl zpracovat.

Hlavní část výstupu představuje hledané sjednocení. Je výsledkem spojení obou vstupních řádků, přičemž program odstranil všechny duplicity a čísla seřadil od nejmenšího po největší. Údaj „Počet prvků“ značí délku výsledné posloupnosti a umožňuje jednoduše zjistit počet unikátních prvků v obou řádcích.

8 Průběh práce

Na začátku práce bylo potřeba se seznámit s hlavními omezeními zadání, které zakazuje používání vestavěných množin. Následně bylo třeba se zamyslet nad algoritmickou podstatou problému. Zvažováno bylo naivní porovnání každého prvku s každým, což bylo pro vysokou časovou náročnost zamítnuto. Bylo poté zvoleno využití vlastností seřazeného pole, kde se duplicity řadí k sobě, což umožňuje efektivnější řešení problému.

Největší výzvou bylo správné ošetření vstupních dat. Problematické byly vstupní soubory s mezerami navíc nebo s prázdnými řádky. Bylo třeba zajistit robustnější načítání souborů. Správnost řešení byla ověřena na náhodně ručně vytvořených souborech, ve kterých byly různé počty duplicit, aby bylo ověřeno, zda vše funguje bezchybně.

9 Možná vylepšení

Program vyhovuje požadavkům zadání, avšak existuje prostor pro vylepšení a zdokonalení. Z hlediska uživatelského komfortu by bylo vhodné zpracování argumentů příkazového řádku,

což by umožnilo specifikovat název vstupního souboru. Dále by bylo možné rozšířit aplikaci o podporu jiných datových typů, například textových řetězců nebo desetinných čísel, což by vyžadovalo úpravu parsovací logiky.